

설계대로 구현하고, AI를 통제하다

의료·자동차 보험 시스템

누가 사용하나 — 세 주체



고객

고객

보험에 가입하고
청구·납부한다



직원

보험사 직원

가입과 지급을
심사한다



시스템

시스템

계약·고지서·지급을
자동 처리한다

어떤 보험을 다루나

의료보험

아프면 진료비를 돌려받는다

자동차보험

사고가 나면 보상받는다

이 서비스는 두 가지 보험을 다룹니다.

의료보험이란

아프면 병원비를 돌려받는 보험

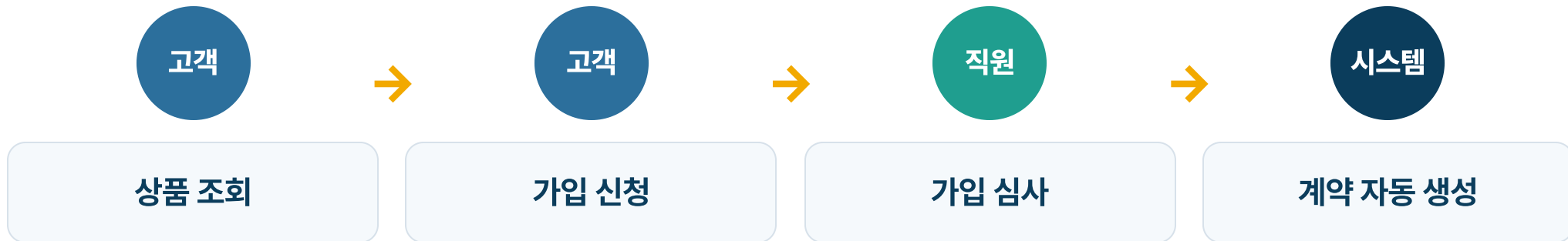
- 병원비를 청구하면 보험금을 계좌로 입금해 준다
- 청구액이 100만 원보다 적으면 — 묻지 않고 바로 입금 (예: 30만 원 청구 → 즉시)
- 100만 원 이상이면 — 직원이 확인한 뒤 입금

자동차보험이란

| 사고가 나면 보상받는 보험

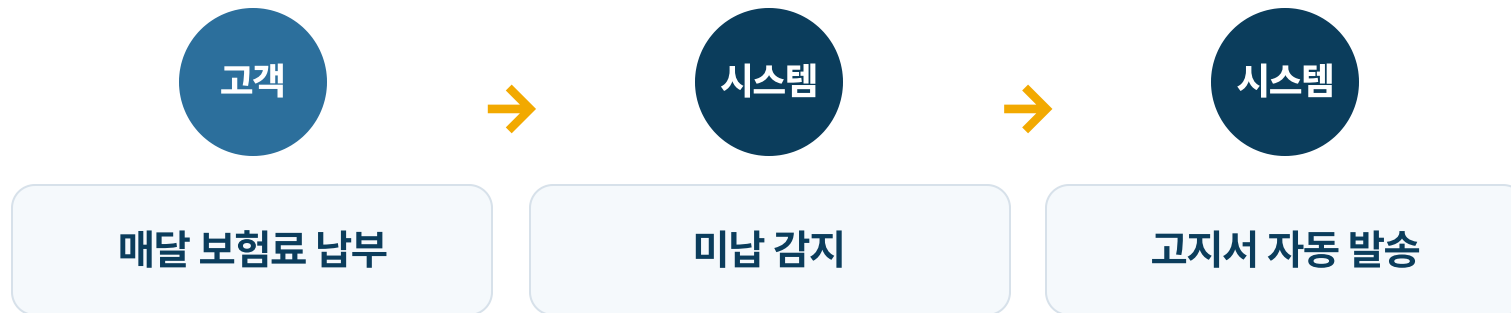
- 사고를 접수하면 접수번호가 나온다
- 직원 한 명이 그 사고를 맡아(배정) 보상 금액을 정한다
- 정해진 금액이 계좌로 입금된다

여정 ① — 가입



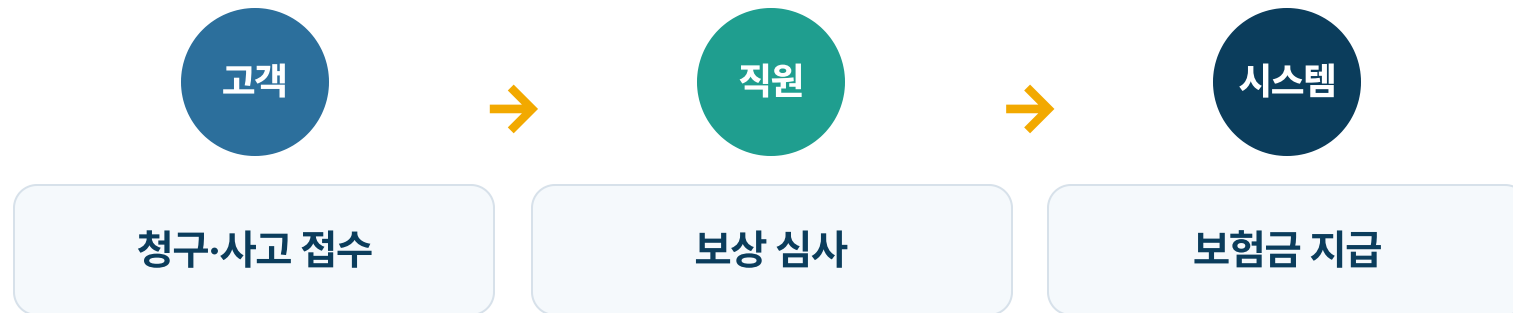
직원이 승인하면, 같은 처리 안에서 계약이 곧바로 만들어집니다.

여정 ② — 납부와 미납



넬 회차보다 납부가 밀리면 미납으로 잡히고, 매일 아침 시스템이 고지서를 보냅니다 (30일 넘으면 해지 예고).

여정 ③ — 보상과 지급



병원비는 100만 원 미만이면 즉시, 이상이거나 자동차 사고면 직원 심사를 거쳐 지급됩니다.

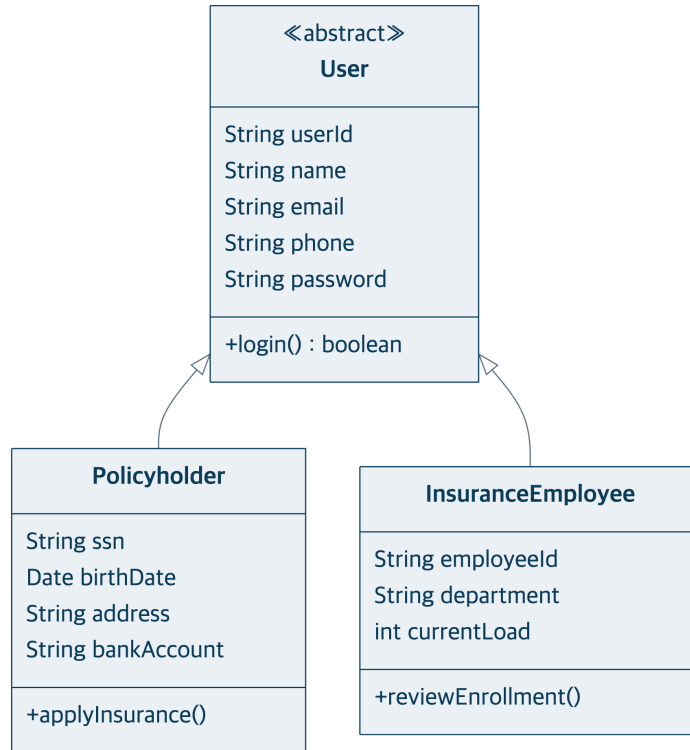
여섯 개의 도메인으로 설계했다

● 사용자 ● 상품 ● 계약 ● 청구 ● 심사 ● 사고이력

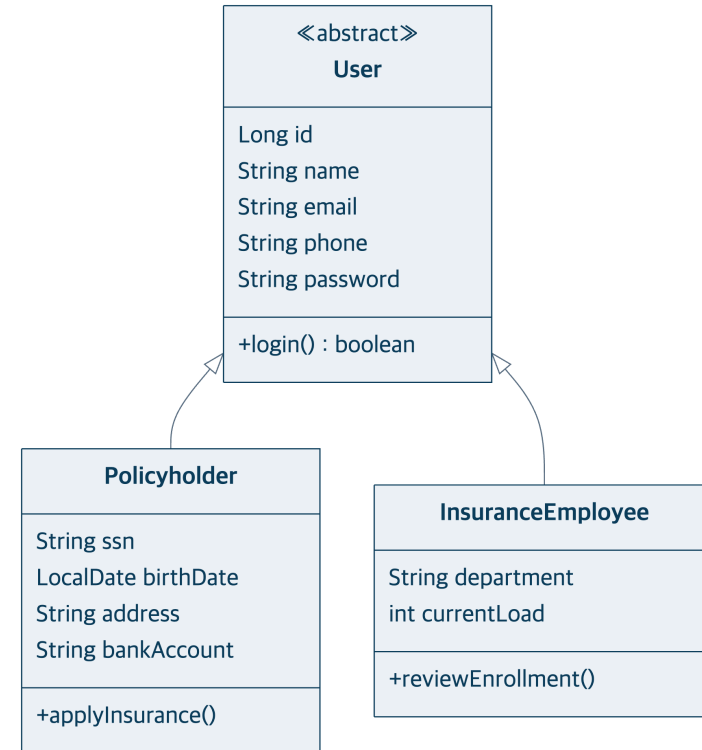
이 여섯 도메인을 직접 클래스 다이어그램으로 설계했고, 지금부터 하나씩 코드와 비교합니다.

설계 = 구현 — 사용자

내가 설계



코드 역설계



= 같은 점

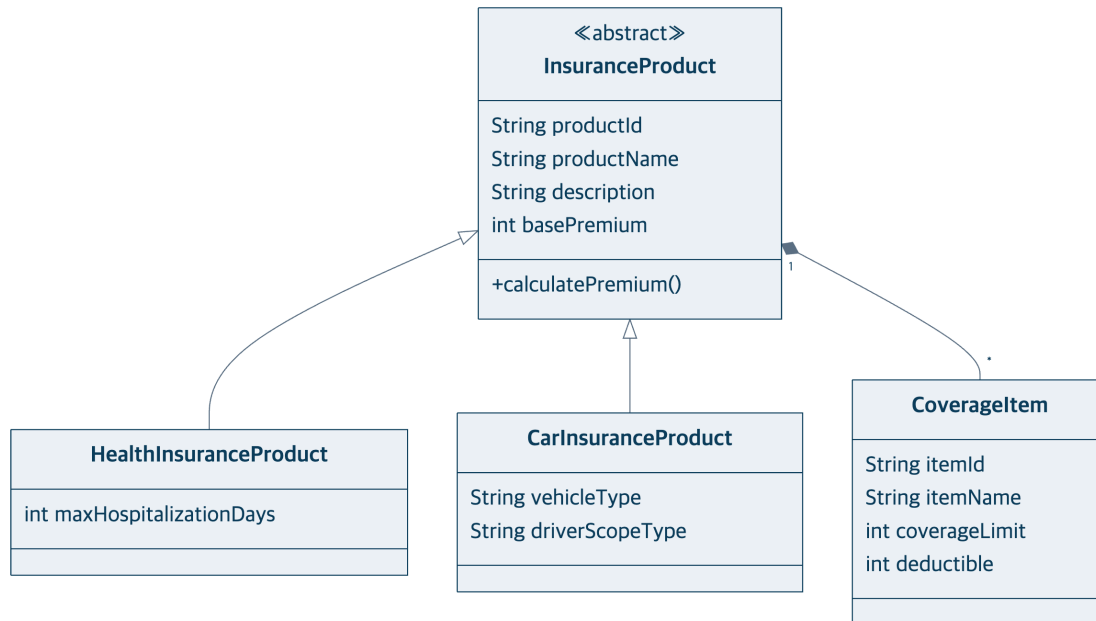
상속(User→Policyholder·Employee)과 모든 필드가 동일.

≠ 다른 점

userId(String) → id(Long), birthDate Date → LocalDate.

설계 = 구현 — 상품

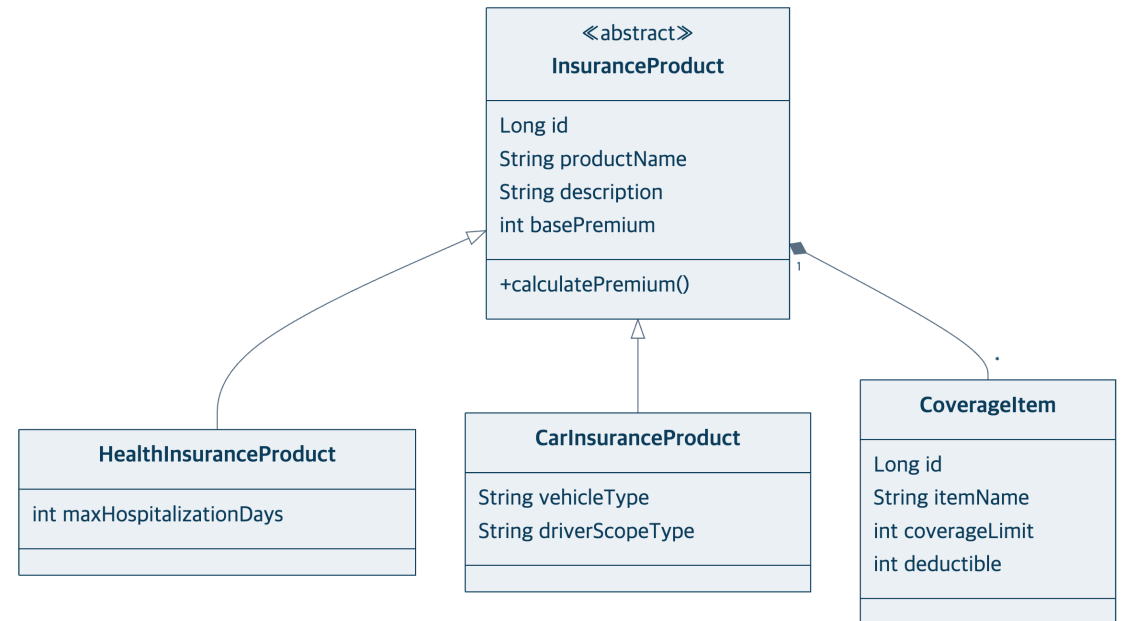
내가 설계



= 같은 점

상품 상속과 CoverageItem 합성 관계가 동일.

코드 역설계

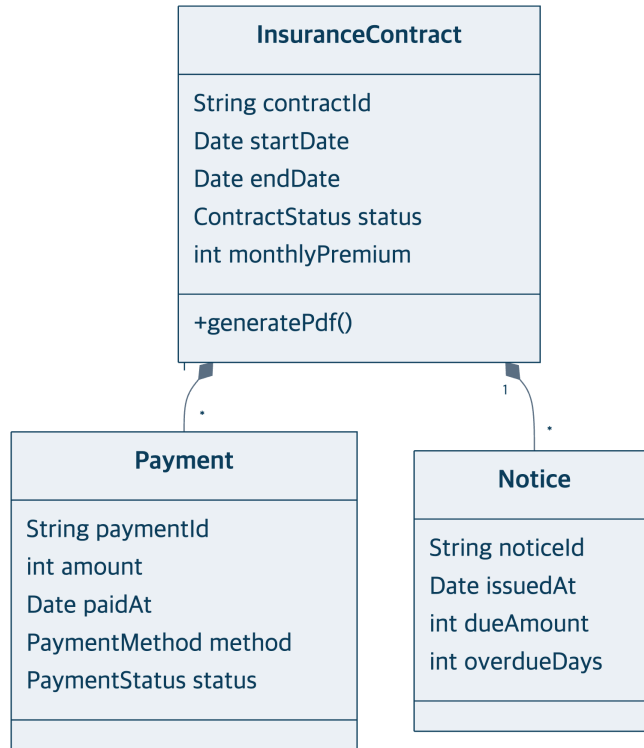


≠ 다른 점

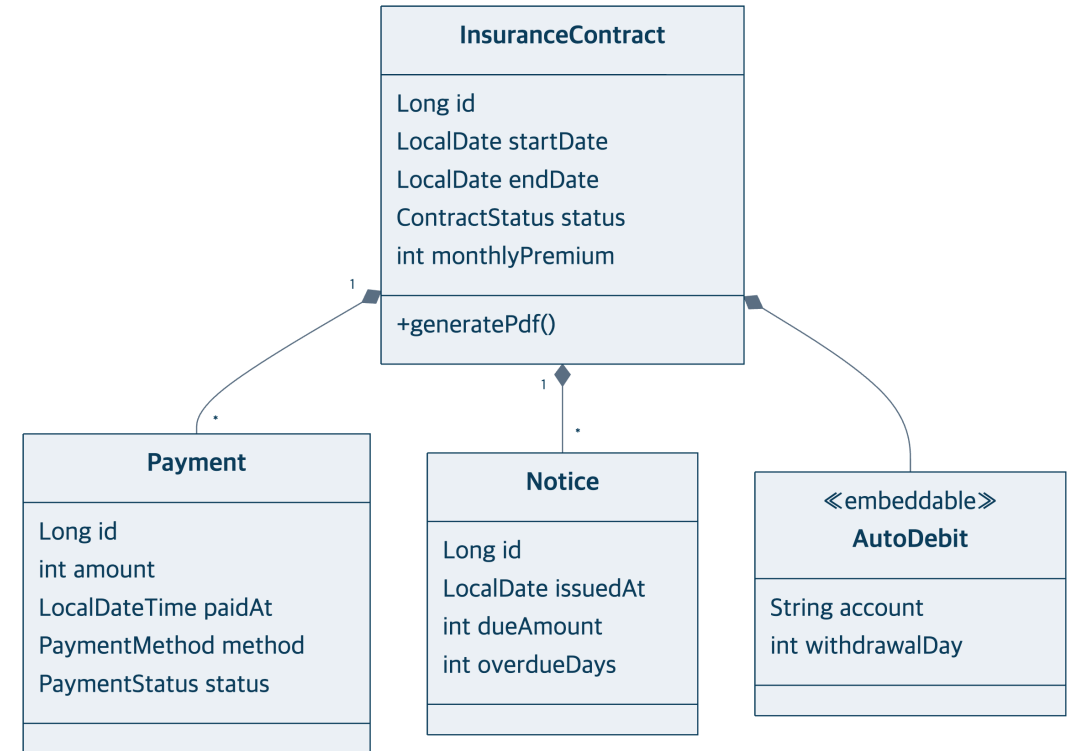
`productId.itemId(String) → id(Long)`.

설계 = 구현 — 계약

내가 설계



코드 역설계



= 같은 점

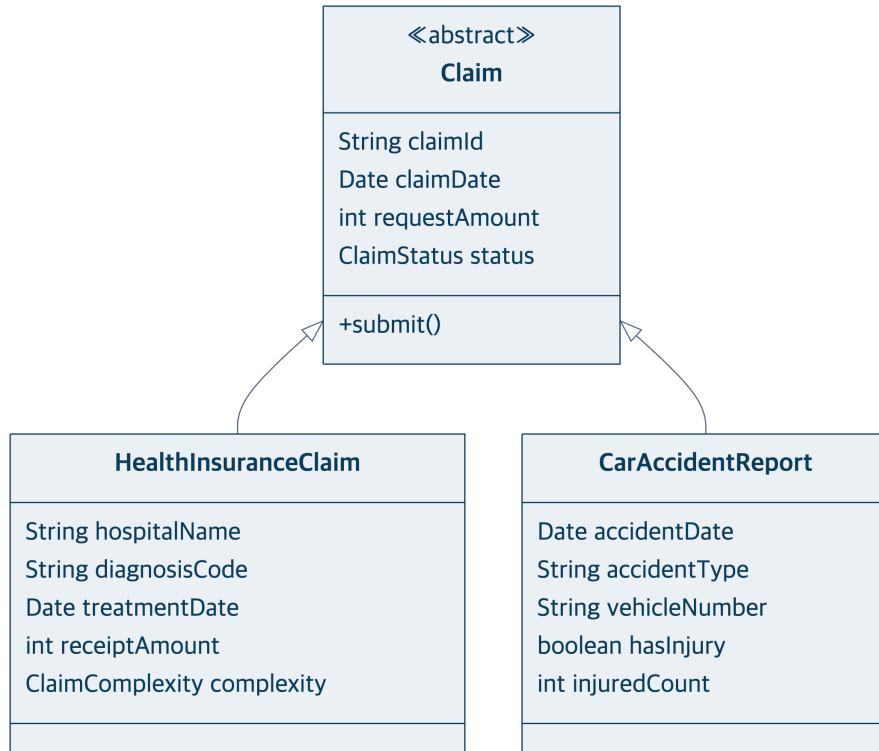
계약-납부-고지 합성 구조가 동일.

≠ 다른 점

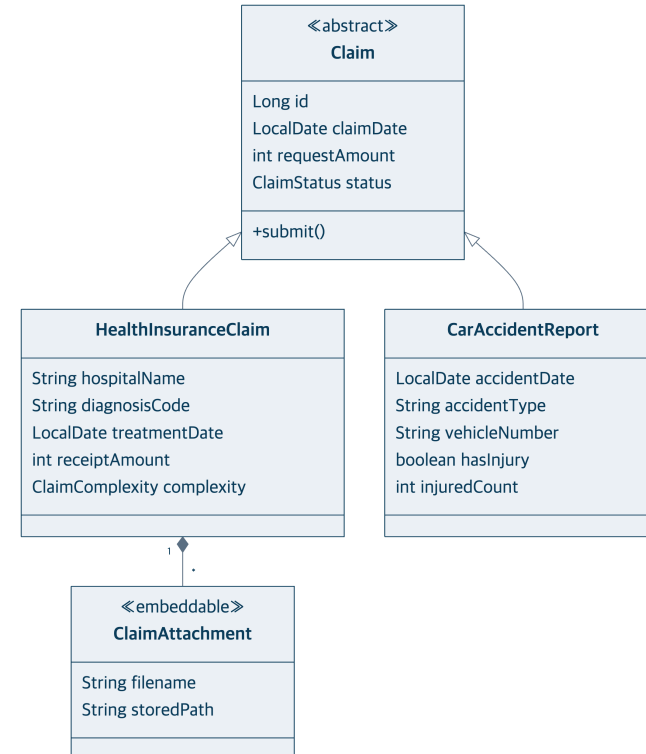
자동이체 정보 AutoDebit 추가, 날짜 타입 변경.

설계 = 구현 — 청구

내가 설계



코드 역설계



= 같은 점

Claim 상속(의료/자동차)과 필드가 동일.

≠ 다른 점

첨부파일 ClaimAttachment 추가, ClaimStatus 4값 → 6값(송금 결과 추적).

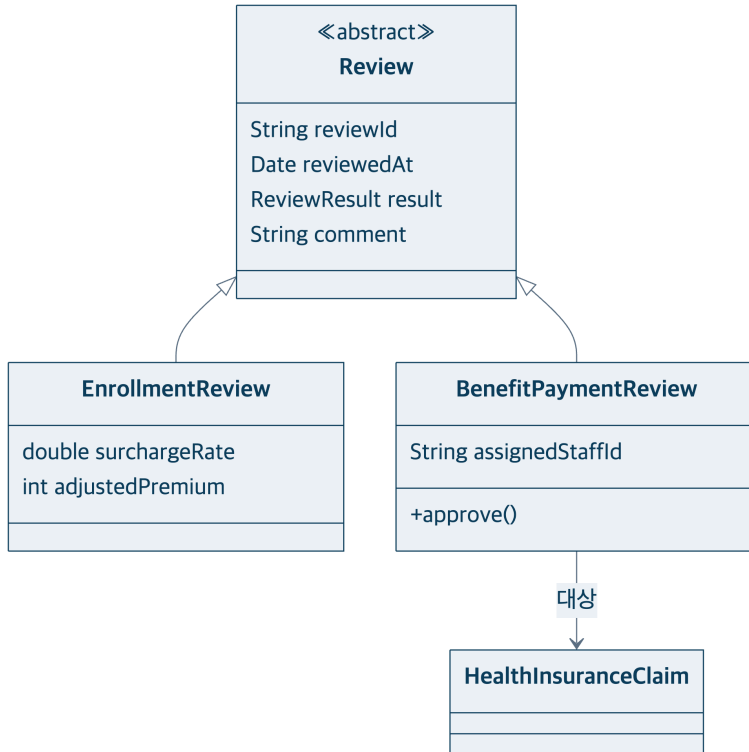
설계 = 구현 — 청구 상태값 (코드)

```
public enum ClaimStatus {  
    PENDING, IN_REVIEW, APPROVED, REJECTED, // 설계 4값  
    COMPLETED, FAILED                     // 구현 추가: 송금 성공·실패 (ADR 0007)  
}
```

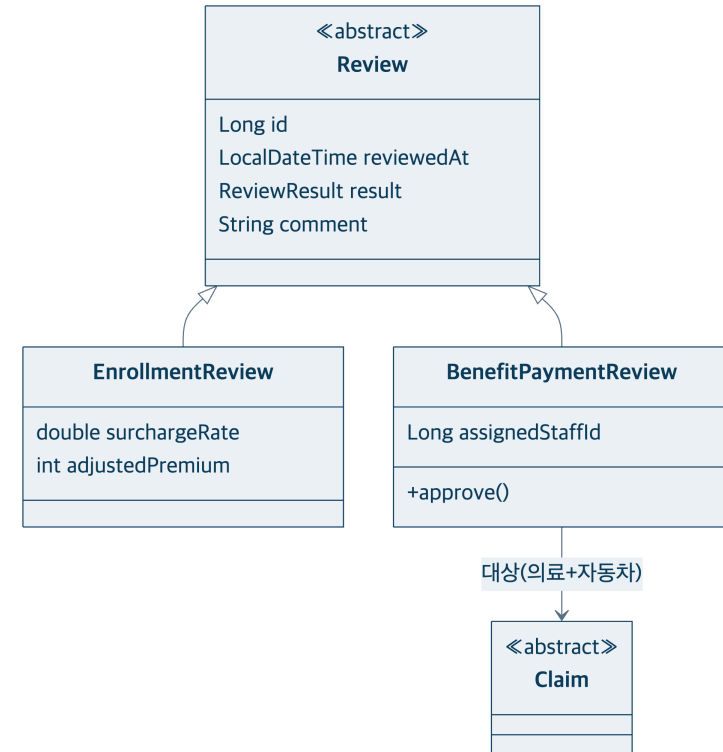
송금 결과까지 추적해야 해서 두 값을 더했습니다.

설계 = 구현 — 심사

내가 설계



코드 역설계



= 같은 점

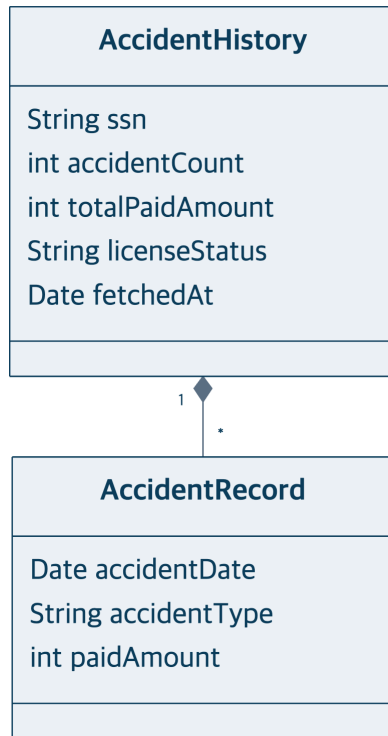
Review 상속(가입심사/지급심사) 구조가 동일.

≠ 다른 점

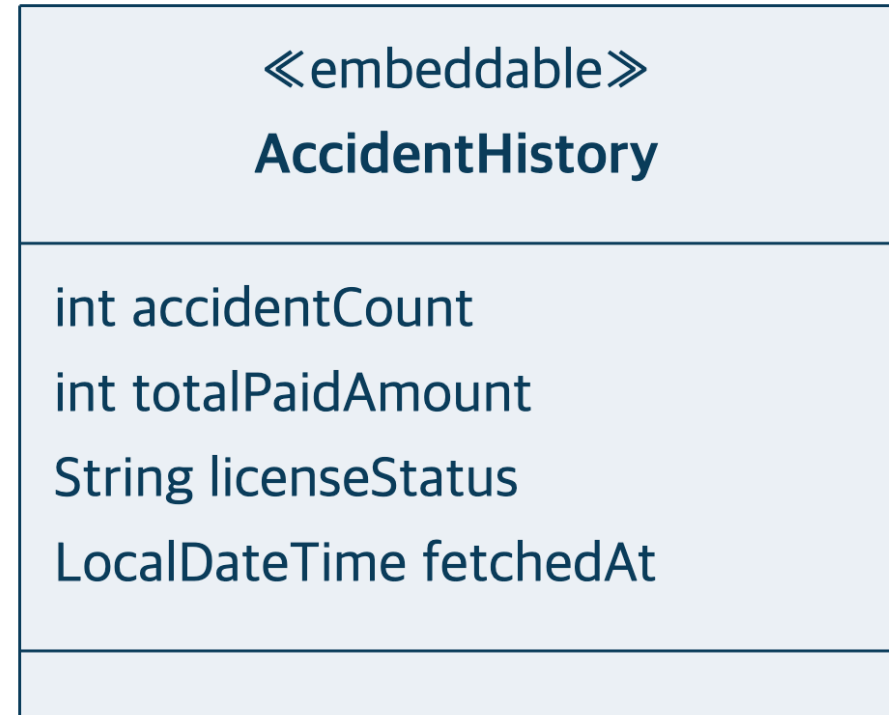
지급심사 대상이 의료청구 → Claim(의료+자동차)으로 확대 (ADR 0009).

설계 = 구현 — 사고이력

내가 설계



코드 역설계



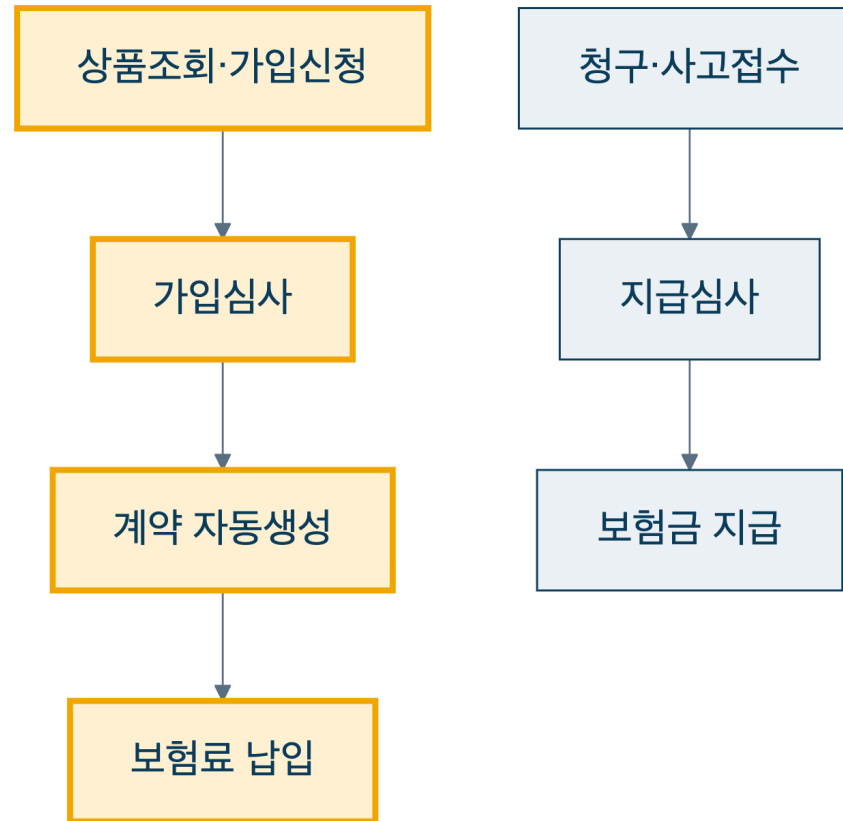
= 같은 점

사고 집계 정보(건수·지급액·면허상태)를 동일하게 보유.

≠ 다른 점

외부 연동이 더미라 개별 AccidentRecord 제거, 값 객체로 단순화.

유스케이스도 설계대로 동작한다



설계한 유스케이스가 데모에서 그대로 동작합니다 (노란 경로).

AI를 어떻게 썼는가

사람(설계자)

다이어그램 설계 · 결정(ADR) · 리뷰

AI

구현 · 테스트 · 리팩토링

설계

다이어그램



취조

모델 대조



계획



TDD 구현

AI



리뷰

설계와 결정은 사람이, 구현은 AI가 했습니다.

AI를 쓰며 생긴 문제와 해결

① 성급한 추상화

실익 없는 계층 분리 → 되돌림

② 명세와 불일치

자동 배정 → 수동 배정으로 정정

③ 계층 패턴 이탈

점검으로 식별·관리

④ 용어·도메인 오염

용어집·ADR로 사전 차단

AI는 빠르지만, 방향은 사람이 잡았습니다.

결론

설계한 그대로 구현했고, 그 과정에서 AI를 통제했습니다.

감사합니다.